Strict Multi-Agent Evaluation Design for OTT-Agentic-AI-News-Broadcaster

1. Context and goals

The current project implements a LangGraph-based multi-agent pipeline that converts a news article URL into a short TV-style broadcast screenplay with narration, segment-wise visuals, and tags.[^1]
The HR brief explicitly requires this system to focus on planning, orchestration, and evaluation, not on video or TTS itself.[^1]
The user has drafted an internal plan to move from a single coarse reviewer into strict per-agent rubrics, deterministic checks, and a full evaluation trace, but the implementation is still pending.[^2]

The goal of this report is to design **industry-style, domain-specific and prompting-oriented evaluation metrics** (≈20–25 metrics per agent) plus a retry-and-trace architecture so that mediocre generations **fail in the first 1–2 attempts** and only genuinely strong packages pass.
The design should align with modern LLM evaluation practice (faithfulness, hallucination detection, instruction-following, etc.) and be realistic for a newsroom-grade broadcast workflow.[3][4][5][6]

2. Current pipeline and weaknesses

2.1 High-level workflow

From the repository and LangGraph screenshots, the current workflow is:

- `journalist_agent` : scrapes article HTML (tiered scraper) and asks an LLM to produce a refined title, story, and facts as free text.
- `editor_agent` : takes the journalist output and writes a 160–320-word broadcast narration (1–2 minutes) as plain text.
- `visualizer_agent` : segments the narration into 3–5 JSON segments with timing, layout, and either source image URL or AI visual prompt.
- `tagger_agent` : in parallel, segments narration into JSON `segment_tags` (headline, subheadline, top tag) without explicit coordination with the visualizer.
- `reviewer_agent` : LLM-as-judge that scores the entire package on 5 coarse metrics and returns a single `failure_type` string (editor, visualizer, tagger) plus `status` .

2.2 Code-level gaps

The plan.md already documents several architectural gaps; code inspection confirms these issues and adds a few more.[^2]

Area	Current behavior	Problem for strict evaluation
Journalist	Single LLM call with qualitative prompt; only guard is checking for error substrings and minimum raw text length.	No faithfulness check to the scraped article, no coverage scoring, no structure constraints; potential hallucinations or missing key facts can slip through.
Editor	Writes narration using instructions for length, tone, and structure but returns raw text only; no deterministic enforcement of word count or structure; ignores any previous review feedback.	Domain quality (hook, beats, close) and prompting quality (instruction following) are not quantitatively measured before moving to visual packaging.
Visualizer	Independently breaks script into segments and returns JSON with timing and layouts.	No deterministic validation of JSON, timecode monotonicity, total duration, or alignment with `tagger` output; failures are only caught (maybe) by reviewer LLM.
Tagger	Independently segments script into headlines/subheadlines/tags JSON.	No guarantee that number of `segment_tags` equals number of visual segments or that tags truly reflect narration beats.
Reviewer	Single LLM judge with 5 coarse scores and one `failure_type` string; increments `iterations` when failing.	No per-agent metric breakdown, no explicit domain vs prompting separation, fragile routing on natural-language `failure_type`, and no historical trace of improvements.
Global	`state.py` has minimal fields (`url` , `iterations` , etc.) and no evaluation trace or	Cannot audit what changed between retries, which metric

Area	Current behavior	Problem for strict evaluation
	last-feedback fields.[^2]	improved, or why a package finally passed.

These gaps match findings from recent work on LLM evaluation and hallucination: high-level quality scores without fine-grained rubrics correlate poorly with human judgment, especially for faithfulness and structure-sensitive tasks like news summarization.[3][4][^7]

3. Evaluation principles from recent research

Modern LLM evaluation in news-like tasks emphasizes three clusters of metrics:[3][4][5][8][^6]

1. **Faithfulness / factual consistency**
   - Outputs must not invent entities, numbers, or claims not supported by the source article.
   - Both intrinsic (within-summary contradictions) and extrinsic (vs. article) hallucinations matter.[4][7]
2. **Relevance, coherence, and structure**
   - Strong correlation between human judgment and metrics measuring relevance of included facts, logical flow, and summary structure in news summarization studies.[^3]
   - News broadcasts especially depend on a strong hook, smooth mid-story progression, and a clear close.
3. **Instruction following and format validity**
   - Enterprise LLM evaluation frameworks (Braintrust, OpenLayer, etc.) include prompt adherence, schema validity, and safety as first-class metrics.[5][6]
   - For automation, models must reliably obey JSON schemas and length/format constraints.[^9]

In addition, emerging techniques like Recursive Language Models (RLM) treat prompting as an environment where the model can iteratively decompose, verify, and refine outputs before finalizing.[10][11]
The multi-agent LangGraph in this project already moves in that direction; the missing ingredient is a **structured, config-driven reward model** encoded as evaluation rubrics instead of ad hoc checks.

4. Metric taxonomy: domain vs prompting

The user and HR want each agent to be evaluated on **two orthogonal axes**:

- **Domain metrics** – "Does this feel like real TV news?" (journalism, storytelling, visual grammar, newsroom ethics).
- **Prompting / craft metrics** – "Does the agent obey format, constraints, and safety?" (instruction following, schema, hallucination risk, robustness).

For each agent, this report defines ≈20–25 metrics, split into domain and prompting groups, with the expectation that some are programmatic and some are LLM-judged.
This aligns with the plan to store these metrics in `config/evaluation_rubric.yaml` with weights and critical flags.[^2]

4.1 Global policy parameters

Global evaluation policy should live in `config/evaluation_policy.yaml` and define:[^2]

- `max_graph_iterations` : 3 (so typical flows see 1–2 retries; 3rd is last attempt).
- `agent_max_retries` : e.g., journalist: 2, editor: 2, visualizer: 2, tagger: 2.
- `min_score_per_metric` : 3 on a 1–5 scale; any critical metric below 4 causes hard fail.
- `overall_pass_threshold` : weighted average ≥ 4.3 for each agent and ≥ 4.5 for cross-package coherence.
- `retry_strategy` : choose retry targets based on failing metrics; avoid unnecessary retries when scores are already excellent.

These thresholds are deliberately **harsh** so that weak outputs fail early and only newsroom-grade packages pass.

5. Metrics for the Journalist Agent

The journalist transforms raw scraped HTML into a structured representation of the article (refined title, core story, key facts).
This is where faithfulness and coverage must be ironclad.

5.1 Domain metrics (≈12)

1. **Source faithfulness** (critical)
    - Judge whether all claims in the refined story are supported by the scraped article text; penalize any invented entities, numbers, or quotes.
2. **Key fact coverage** (critical)
    - Percentage of core 5–7 facts (who, what, where, when, why, how, impact) present in the refined story.

3. **Lead clarity**
    - Whether the refined title and first paragraph clearly communicate the main event.
4. **Context and background**
    - Inclusion of essential context (location, stakes, affected population) without drifting into unrelated commentary.
5. **Balance and neutrality**
    - No strong opinionated language; balanced framing consistent with professional news tone.
6. **Temporal accuracy**
    - Tenses and timeline are consistent with the article (past vs ongoing vs future developments).
7. **Numerical fidelity**
    - Numbers (casualties, money, counts) match the article exactly.
8. **Named entity accuracy**
    - Proper names (people, places, organizations) spelled correctly and not hallucinated.
9. **Quote integrity**
    - Direct quotes are either copied exactly or clearly paraphrased; no fabricated quotes.
10. **Story completeness**
    - The refined story captures beginning, main developments, and current status where applicable.
11. **Redundancy control**
    - Avoids repeating the same fact multiple times.
12. **Language and grammar quality**
    - Professional written English with minimal grammatical errors and no informal slang.

5.2 Prompting / craft metrics (≈10)

13. **Instruction adherence**
    - Output follows exactly the requested structure: `TITLE:` , `STORY:` , `FACTS:` blocks.
14. **Length control**
    - `STORY` section stays within a configured word-count window (e.g., 350–700 words) suitable for downstream use.
15. **FACTS structure quality**
    - `FACTS` section lists discrete, machine-parseable bullets or numbered facts (no paragraphs).
16. **Forbidden phrases**
    - Avoids meta-comments like "this article is about" or "as an AI language model".
17. **Encoding and character sanity**
    - No broken encodings, HTML artifacts, or unreadable characters.
18. **Safety and compliance**

- No hate, explicit violence details beyond newsroom standards, or personal PII; aligned with typical newsroom safety policies.[8][6]

19. **Deterministic schema validity**
    - Programmatic check that the output matches a structured schema (e.g., parse via regex/JSON) without missing fields.
20. **Language detection**
    - If article language differs from configured broadcast language (e.g., English), either translation is explicit or agent fails with a clear message.
21. **Error-handling discipline**
    - Error states (e.g., 404 or thin content) use structured error outputs and do not bleed partial hallucinated summaries into downstream state.
22. **Latency budget respect (optional)**
    - Enforce max tokens per call and response time constraints; long responses that risk timeouts are penalized.

6. Metrics for the Editor Agent

The editor writes the actual 60–120s narration that the anchor will speak.
This is where story arc, pacing, and spoken-language quality matter.

6.1 Domain metrics (≈12)

1. **Hook strength** (critical)
    - First 1–2 sentences capture attention while accurately reflecting the story.
2. **Story structure** (critical)
    - Clear beginning, middle, and ending; ending provides closure, next steps, or recap.
3. **Beat clarity**
    - Distinct 3–4 story beats (e.g., event, response, impact, what-next) that segment cleanly for visuals.
4. **Factual consistency with journalist facts** (critical)
    - No new facts beyond `FACTS` list or scraped article; no contradictions.
5. **Tone appropriateness**
    - TV newsroom tone: professional, calm yet engaging; no jokes or memes for serious topics.
6. **Pacing suitability**
    - Word count and sentence rhythm match spoken delivery (no 60-word monsters).
7. **Clarity and simplicity**
    - Sentences are easy to read aloud; limited nested clauses or jargon.
8. **Avoidance of sensationalism**

- No exaggerated adjectives or misleading framing compared to source.

9. **Audience orientation**
    - Uses second-person or general framing appropriately ("Tonight we" / "Viewers"), depending on style guide.
10. **Redundancy and repetition**
    - Minimal repetition between beats; each beat contributes new information.
11. **Transition smoothness**
    - Logical connectors between beats ("Meanwhile", "At the same time", "Looking ahead").
12. **Pronunciation friendliness**
    - Avoids tongue-twister phrasing and overlong lists that will sound awkward in TTS.

6.2 Prompting / craft metrics (≈10)

13. **Word-count adherence** (critical)
    - Deterministically computed; narration between 160 and 320 words as specified.
14. **No bullet lists**
    - Programmatic check that output is continuous narration, not bulleted or numbered lists.
15. **Section presence**
    - Optional: enforce markers for hook, middle, close so visualizer can align segments (e.g., `HOOK:`, `BODY:`, `CLOSE:`) or at least recognizable paragraph boundaries.
16. **Instruction adherence**
    - Script follows tone and structure instructions; evaluation via LLM judge with rubric.
17. **No meta-text**
    - Bans phrases like "In this script" or "The following narration".
18. **Profanity / safety**
    - Simple toxic keyword filters plus LLM moderation; needed for broadcast compliance.[8][6]
19. **Format stability across retries**
    - On retries, editor remains consistent in structure (paragraph separation) so downstream diffing is meaningful.
20. **Language style match**
    - Optionally check adherence to a style guide (e.g., Indian English vs US English) via LLM-as-judge.
21. **Token budget adherence**
    - Response tokens within configured limit to protect latency.
22. **Prompt-responsiveness**
    - When given metric-level feedback (e.g., "hook too weak"), new iteration demonstrates measurable improvement on those specific metrics.

7. Metrics for the Visualizer Agent

The visualizer turns narration into timed segments with layouts and visuals.
This is the most "TV-specific" component, where real newsroom standards around visual rhythm and chyron readability must be encoded.

7.1 Domain metrics (≈13)

1. **Segment count appropriateness** (critical)
   - 3–5 segments for 60–120s content; no 1-segment or 10-segment extremes.
2. **Beat alignment** (critical)
   - Segment boundaries align with editorial beats (e.g., hook vs details vs impact vs wrap-up).
3. **Timecode coverage and continuity** (critical)
   - Start at `00:00`, end within 60–120 seconds, and no gaps or overlaps between segments.
4. **Layout variety with purpose**
   - Uses different layouts (anchor left, full-screen visual, etc.) to support narrative, not random switching.
5. **Visual-story match** (critical)
   - For each segment, chosen source image or AI visual prompt accurately depicts the narrated content.
6. **Use of source images**
   - Prioritizes real article images where available and contextually relevant; AI visuals used as support, not replacement.
7. **Emotional tone alignment**
   - Visuals respect emotional seriousness of the story (e.g., avoid glamorous effects for tragedies).
8. **Avoidance of misleading visuals**
   - No visuals that could be mistaken for real footage when they are AI-generated, unless clearly stylized.
9. **Transition logic**
   - Reasonable transitions (`cut` , `crossfade` , `slide`) at segment boundaries; no abrupt mid-sentence hard cuts.
10. **Chyron readability**
    - Layout choices leave space for headlines/subheadlines; avoid cluttered visuals behind text.
11. **Shot diversity**
    - Mix of anchor-in-studio vs B-roll style visuals; avoids single static angle for entire piece.
12. **Crisis/no-crisis treatment**

- For sensitive stories (disasters, violence), visual prompts should avoid graphic details per newsroom guidelines.[^8]

13. **Localization appropriateness**
    - For region-specific stories, visuals reflect correct geography and cultural context.

7.2 Prompting / craft metrics (≈10)

14. **JSON schema validity** (critical)
    - Deterministic validation: `video_duration_sec`, list of segments, each with `segment_id`, `text`, `start_time`, `end_time`, `layout`, `source_image_url`, `ai_support_visual_prompt`.
15. **Time format correctness**
    - `MM:SS` or `HH:MM:SS` with zero padding; no malformed strings.
16. **Monotonic timecodes**
    - Programmatic check that `start_time < end_time` and segments are ordered.
17. **Duration vs word-count consistency**
    - Approximate speech-rate check (e.g., 2–3 words per second) to ensure segment times fit narration text length.
18. **Null-handling discipline**
    - Exactly one of `source_image_url` or `ai_support_visual_prompt` must be non-null; never both null.
19. **Prompt completeness**
    - AI visual prompts contain subject, setting, style ("realistic news-style"), and region; avoid vague 3-word prompts.
20. **Deterministic cross-check with tagger**
    - Segment count equals number of `segment_tags`; index-aligned.
21. **No meta-text in prompts**
    - Visual prompts describe the scene directly, not instructions like "generate an image".
22. **Safety filters for prompts**
    - Reject prompts violating visual safety policy (graphic gore, hate symbols).
23. **Deterministic size limits**
    - Limit prompt length to e.g. 120 tokens for cost and latency reasons.

8. Metrics for the Tagger Agent

The tagger creates headlines, subheadlines, and top tags per segment.
This is where newsroom-style "punch" is measured.

8.1 Domain metrics (≈12)

1. **Segment alignment** (critical)
   - Headlines correspond to the same beats as visual segments; enforced via count and content checks.
2. **Headline punch** (critical)
   - Strong, concise line capturing main point in ≤ 30 characters.
3. **Subheadline clarity**
   - Adds useful detail in ≤ 50 characters; no full sentences.
4. **Tag appropriateness** (critical)
   - `BREAKING`, `LIVE`, `DEVELOPING`, `UPDATE`, etc., reflect actual story status, not random labels.
5. **Avoidance of clickbait**
   - No misleading exaggerations or vague "You won't believe" style phrases.
6. **Consistency with narration**
   - No contradictions with spoken text.
7. **Style guide adherence**
   - Capitalization, tense, and punctuation match a configured style (e.g., title case, no periods).
8. **Diversity across segments**
   - Headlines evolve with story beats instead of repeating the same phrase.
9. **Localization of wording**
   - Region names, city names, and cultural terms used correctly.
10. **Sensitivity for tragedies**
    - Respectful wording for loss-of-life or sensitive events.
11. **Screen-fit**
    - Character limits and word lengths designed to avoid overflow on typical broadcast lower thirds.
12. **Redundancy control**
    - Subheadline adds new aspect, not simply repeating headline.

8.2 Prompting / craft metrics (≈10)

13. **JSON schema validity** (critical)
    - `segment_tags` list with each object having `headline`, `subheadline`, `top_tag`.
14. **Character-limit adherence**
    - Enforce max 30 chars headline, 50 chars subheadline programmatically.
15. **Tag vocabulary adherence**
    - `top_tag` must be one of allowed values (config-driven) unless explicitly configured as "OTHER".

16. **No meta-text**
    - Avoids phrases like "Headline:" or "Subheadline:" in the text itself.
17. **Safe language**
    - No slurs, hate speech, or explicit terms.
18. **Response-format stability**
    - On retries, remains consistent JSON; no stray text before or after JSON.
19. **Deterministic segment-count equality**
    - Match `len(segment_tags)` to `len(segments)` from visualizer; fail early if mismatch.
20. **Prompt responsiveness**
    - On feedback (e.g., "headline too generic"), measurable improvement in those scores.
21. **Language correctness**
    - Grammar and spelling correct despite tight character limits.
22. **No emoji / informal markers**
    - Enforce newsroom style; e.g., no " 🔥 " or "LOL".

9. Cross-package and Reviewer metrics

The existing reviewer currently scores 5 dimensions across the whole package and picks a single failing agent.
In the new design, the reviewer becomes an orchestrator that runs per-agent LLM judges and aggregates deterministic checks.
Additional **cross-package** metrics ensure the whole broadcast behaves like a coherent TV segment.

9.1 Cross-package domain metrics (≈8)

1. **Global story coherence** (critical)
    - Title, narration, visuals, and headlines all tell the same story without contradictions.
2. **Temporal alignment**
    - Narrator references ("earlier today", "tonight") match context; visuals do not show night when narration says "morning".
3. **Narration-to-visual rhythm**
    - Dialog density vs visual change frequency feels natural (no rapid cuts during dense information bursts).
4. **Duration fit** (critical)
    - Total `video_duration_sec` within allowed window; final segment not unnaturally long or short. [^1]
5. **Coverage completeness**
    - Major article facts appear somewhere in the package; nothing critical is missing.[^3]

6. **Redundancy across modalities**
    - Acceptable degree of repetition between narration and headlines (for reinforcement), but not verbatim duplication.
7. **Overall engagement**
    - Subjective measure of how watchable the piece is; ties to HR's "superman" expectation.
8. **Ethical and safety compliance**
    - No biased framing, harmful stereotypes, or policy violations.[8][6]

9.2 Cross-package prompting metrics (≈7)

9. **No schema drift across retries**
    - All agents maintain the same output schema version so downstream systems are stable.
10. **Traceability and reproducibility**
    - Evaluation trace includes model versions, prompts, seeds/temperature (where available), and decision logs.
11. **Deterministic evaluation**
    - LLM judges run with temperature 0, reference stable rubric text, and output JSON with numeric scores and evidence snippets.[2][4]
12. **Policy-respecting routing**
    - Retry decisions obey `evaluation_policy.yaml` instead of ad hoc conditions.
13. **Cost and latency budgeting**
    - Limit per-round judge token usage; skip re-evaluating metrics whose input has not changed.
14. **Fallback behavior**
    - If repeated failures occur, system exits gracefully with clear error instead of looping.
15. **Versioned rubrics**
    - Rubric and policy versions recorded in outputs for audit and reproducibility.

10. Evaluation architecture and data structures

10.1 Rubric configuration files

The following configuration structure can be used, consistent with the internal plan:[^2]

- `config/evaluation_rubric.yaml`
    - Hierarchy: `agent -> metric_id -> {name, group, weight, critical, min_score, evaluator}`.
    - `group` ∈ { `domain`, `prompting` }.
    - `evaluator` ∈ { `code`, `llm`, `hybrid` }.

Example (simplified):

```yaml
journalist:
  faithfulness:
    name: Source faithfulness
    group: domain
    weight: 3.0
    critical: true
    min_score: 4
    evaluator: llm
  coverage:
    name: Key fact coverage
    group: domain
    weight: 2.5
    critical: true
    min_score: 4
    evaluator: llm
  schema_valid:
    name: Structured output validity
    group: prompting
    weight: 2.0
    critical: true
    min_score: 5
    evaluator: code
```

- `config/evaluation_policy.yaml`
  - `max_graph_iterations` , `agent_max_retries` , `overall_pass_threshold` , etc.

10.2 Evaluation modules

A dedicated `evaluation/` package should encapsulate evaluation logic, as already envisioned.[^2]

Recommended files:

- `evaluation/load_rubric.py` – parse and validate YAML (no missing metric IDs).
- `evaluation/deterministic_checks.py` – implement programmatic checks for all `evaluator: code` metrics (JSON validity, word counts, timecodes, alignment, character limits).
- `evaluation/llm_judge.py` – per-agent judge functions that:
  - Take structured state + failed deterministic results.
  - Ask LLM to output a JSON object: `{metric_id: {score: 1-5, evidence: str}}` using temperature 0 and response-format enforcement.[4][5]

- `evaluation/aggregate.py` – combine deterministic and LLM scores into:
  - Per-agent weighted score.
  - `retry_targets` : list of agents needing retry.
  - `blocking_metrics` : list of metric IDs causing fail.
- `evaluation/trace.py` – append evaluation result objects to `state.evaluation_trace` with deltas vs previous round.

10.3 State extensions

`state.py` should be extended with fields:[^2]

- `evaluation_trace: List[EvaluationRound]` – each round: `{round, agent, metrics, pass, retry_targets, notes}` .
- `last_feedback_by_agent: Dict[str, str]` – human-readable bullet list of failing metrics and fix suggestions.
- `raw_article_text` or `scrape_snapshot` – ground truth for journalist/editor faithfulness checks.
- `rubric_version` , `policy_version` – to track configuration.

These additions enable HR and future reviewers to see exactly *what improved where* between retries, which is crucial for a realistic newsroom workflow.

11. LangGraph routing and retries

11.1 Graph changes

The planned high-level graph in plan.md already captures the right structure: journalist → `evaluate_journalist` gate → editor → `visual_packaging_fork` (visualizer + tagger) → `validate_parallel_outputs` → `evaluate_package` → conditional retries.[^2]

Implementation details:

1. **After `journalist_agent`**
   - Call `evaluate_journalist` (deterministic + LLM judge on journalist metrics).
   - If fail and retries remain, route back to `journalist_agent` with `last_feedback_by_agent["journalist"]` appended to the prompt.
   - If unrecoverable (e.g., scraping failure), end early with error artifact.
2. **After `visual_packaging_fork`**
   - Run `validate_parallel_outputs` (deterministic only) to:
     - Check JSON schemas, segment counts, timecodes, and tag–segment alignment.

- If deterministic checks fail, route back to `visualizer` and/or `tagger` with precise error messages; do not call expensive LLM judges yet.

3. **Global evaluation node (`evaluate_package`)**
   - Runs LLM judges for editor, visualizer, tagger, and cross-package metrics in one or multiple calls.
   - Aggregates scores using rubric and decides `retry_targets` .
4. **Conditional routing**
   - If all agents pass and cross-package score ≥ threshold, go to assembler and final output.
   - Otherwise, route only failing agents back, decrementing retry budgets; stop overall when `max_graph_iterations` reached.

11.2 Feedback injection into prompts

For retries to be meaningful, agents must see structured feedback:

- **Journalist prompt** – add a section like:
  - "Previous evaluation feedback: faithfulness=3 (hallucinated casualty numbers), coverage=2 (missing government response). Fix these specifically."
- **Editor prompt** – include bullets from failing metrics (e.g., "Hook too generic; strengthen first sentence", "Ending lacks closure").
- **Visualizer/tagger prompts** – mention mismatch counts or weak headline metrics explicitly.

This follows the same philosophy as RLHF reward-model-based improvements in instruction-following models, but implemented at inference time using deterministic and LLM-based judges.[[12][13][^6]

12. Prompt-level improvements per agent

Besides evaluation, prompts themselves should be upgraded based on modern prompting research and newsroom needs.[[3][9][[5][6]

12.1 Journalist

- Explicitly instruct "Do not invent any facts not present in the article text; if a detail is missing, leave it out instead of guessing."
- Ask for structured JSON output instead of free text; this simplifies deterministic checks.
- Encourage extraction of entities as lists (people, places, orgs) for better alignment tests.

12.2 Editor

- Consider passing structured `FACTS` from journalist instead of full text to reduce noise and discourage hallucinations.
- Require an explicit segmentation of the narration into labelled beats (e.g., `BEAT 1`, `BEAT 2`, etc.) so visualizer and tagger can align.
- Enforce temperature 0 for stability.

12.3 Visualizer

- Add explicit reference to beat labels so each `segment_id` corresponds to a beat rather than arbitrary slicing.
- Emphasize newsroom realism in prompts ("standard TV news broadcast visuals, not cinematic movie style").

12.4 Tagger

- Include examples of excellent vs poor headlines and tags from the HR document to ground the model.
- Clarify that some tags, like `BREAKING`, should be used only when the event is ongoing or very recent.

13. Implementation roadmap

A pragmatic order of implementation, aligned with both HR expectations and technical complexity, is: [^2]

1. **Config and state groundwork**
   - Add rubric and policy YAMLs.
   - Extend `state.py` with evaluation fields.
2. **Deterministic checks**
   - Implement JSON schema validators, word-count checks, timecode checks, and cross-branch alignment before touching LLM judges.
3. **Per-agent LLM judges**
   - Implement `evaluate_journalist`, `evaluate_editor`, `evaluate_visualizer`, `evaluate_tagger`, and `evaluate_cross_package` with temperature 0 and rubric-driven scoring.
4. **Graph wiring and retry policies**
   - Insert evaluation nodes into LangGraph and implement routing based on `retry_targets` and remaining budgets.

5. **Prompt feedback integration**
    - Update agent prompts to ingest `last_feedback_by_agent` and explicitly focus on improving low-scoring metrics.
6. **Reporting and documentation**
    - Add `evaluation_report.json` or embed final metric snapshot into broadcast plan.
    - Write `PROJECT_REPORT.md` summarizing methodology, metrics, limitations, and future work for HR review.

If executed carefully, this architecture will transform the current prototype into a **strict, auditable, newsroom-grade evaluation system** where each agent is held accountable on 20+ metrics, weak outputs naturally trigger retries with targeted fixes, and only genuinely high-quality broadcast plans reach the final stage.

References

1. LOQO_AI_News_URL_to_Dynamic_Broadcast_Screenplay_Generator.pdf - LOQO AI - News URL to Dynamic
   Broadcast Screenplay Generator
   Objective
   Build a multi-agent AI pipeli...
2. plan.md - ---
   name: Strict per-agent evaluation
   overview: "Introduce config-driven, broadcast-domain rubrics...
3. 5.1 Relevance And Coherence...
4. Quantifying Hallucination in Faithfulness Evaluation - In this paper, we investigate automated faithfulness evaluation in guided NLG. We developed a rubric...
5. LLM evaluation metrics: Full guide to LLM evals and key metrics - Evaluation metrics turn subjective AI quality into measurable numbers. Without metrics, you rely on ...
6. LLM evaluation metrics: Complete guide for March 2026 - LLM evaluation metrics covering accuracy, safety, RAG testing, and production monitoring for enterpr...
7. A review of faithfulness metrics for hallucination ... - by B Malin · 2024 · Cited by 31 — Abstract— This review examines the means with which faithfulness h...
8. LLM Evaluation: Frameworks, Metrics, and Best Practices - LLM evaluation is the process of testing and measuring how well large language models perform in rea...
9. LLM Summarization of Large Documents: How to Make It ... - Summarizing text is one of the main use cases for large language models. Clients often want to summa...

10. R.I.P. Basic Prompting - The AI Corner - MIT CSAIL introduces Recursive Language Models (RLMs), a new inference-time approach that lets AI re...
11. MIT's new 'recursive' framework lets LLMs process 10 million tokens ... - Recursive language models (RLMs) are an inference technique developed by researchers at MIT CSAIL th...
12. [PDF] Lecture 10 - MIT OpenCourseWare - *The approach has two main steps: (1) Supervised Fine-Tuning (2) Reinforcement Learning from Human F...
13. Training language models to follow instructions with human feedback - Making language models bigger does not inherently make them better at following a user's intent. For...